

OOP Task

1. Abstract Classes vs Interfaces

What is an abstract class, and what makes a method “abstract” in Python?

An abstract class is a class that is designed to be a base class for other classes. It can contain common attributes and implemented methods, but it can also define methods that subclasses are required to implement. In Python, abstract classes are usually created using the `abc` module, which provides `ABC` and `abstractmethod`.

A method becomes abstract when it is decorated with `@abstractmethod`. An abstract method defines the required method interface for subclasses, but it does not have to provide a complete implementation. Any concrete subclass must implement the abstract methods before it can be instantiated.

What is partial abstraction vs full abstraction?

Partial abstraction means that an abstract class can contain both abstract methods and fully implemented methods. It can therefore provide shared functionality while leaving some behavior to its subclasses.

Full abstraction generally means exposing only the required operations while hiding the implementation details. In languages such as Java, interfaces are commonly used for this purpose. Python does not enforce a separate 'interface' keyword in the same way, but similar designs can be achieved with abstract base classes and protocols.

Does Python have a dedicated interface keyword like Java or C++?

No. Python does not have a dedicated interface keyword like Java. Instead, developers can achieve interface-like behavior mainly through abstract base classes (ABCs) in the `abc` module or through `typing.Protocol`. An ABC is useful when explicit inheritance and required methods are desired, while `Protocol` supports structural typing and is closely related to duck typing.

What happens if you try to instantiate a class that has unimplemented abstract methods?

Python raises a `TypeError`. A class that still has one or more unimplemented abstract methods is considered abstract and cannot be instantiated until those methods are implemented.

2. Duck Typing and Polymorphism in Python

What is duck typing, and where does the name come from?

Duck typing is a programming approach in which the suitability of an object is determined by the methods and behavior it provides rather than by its class or inheritance relationship. The idea is commonly summarized by the phrase: 'If it walks like a duck and quacks like a duck, then it can be treated as a duck.' In Python, this means that an object can be used as long as it supports the operations that the code needs.

How does duck typing let different objects be used interchangeably without inheriting from a common parent?

With duck typing, different and unrelated classes can be used by the same function if they provide the expected method or behavior. The function does not need to check whether the objects inherit from the same parent class.

Dog and Cat do not need to inherit from the same parent class. Both work with `make_speak()` because they provide the `speak()` method.

How is duck typing different from method overriding (polymorphism through inheritance)?

Method overriding usually occurs when a subclass inherits from a parent class and provides its own implementation of a method defined by the parent. The relationship between the classes is explicit.

Duck typing does not require inheritance. The objects can belong to completely unrelated classes; what matters is that they support the required behavior. Therefore, both approaches can provide polymorphic behavior, but they achieve it differently: inheritance-based polymorphism relies on a shared class hierarchy, while duck typing relies on compatible behavior.

Example of duck typing being used instead of strict type-checking:

```
class Printer:
    def print_document(self):
        print("Printing document")

class Screen:
    def print_document(self):
        print("Displaying document")

def process(device):
    device.print_document()

process(Printer())
process(Screen())
```

The `process()` function does not check whether `device` is a `Printer` or `Screen`. It simply calls `print_document()`. As long as the object provides that method, it can be used by the function. This is a simple example of duck typing in Python.

Conclusion

Abstract classes provide a way to define a common structure and required behavior for subclasses, while Python's Protocols and duck typing provide more flexible interface-like designs. Duck typing is especially important in Python because it focuses on what an object can do rather than what type it is. Both abstraction and polymorphism help make OOP programs more organized, reusable, and flexible.